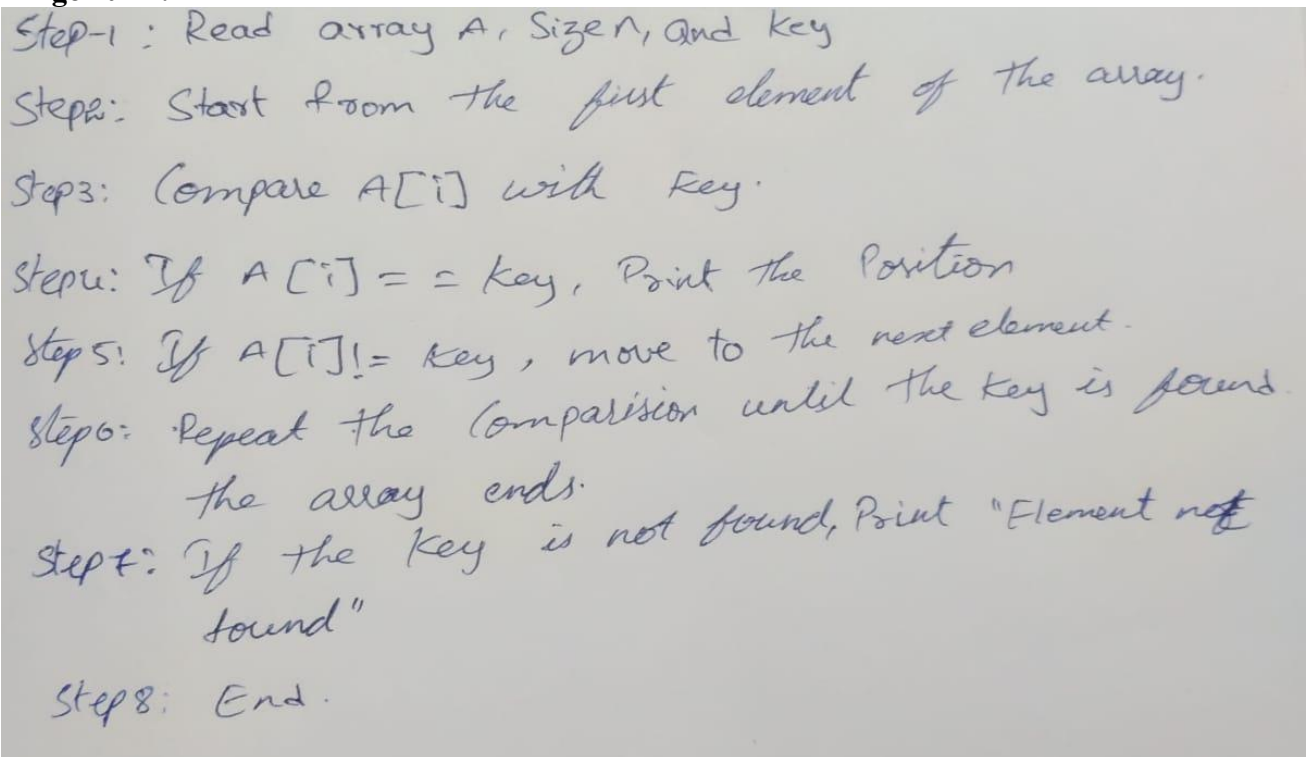


Practical - 2

Aim: To Implementation and Time analysis of linear and binary search algorithm.

1) Linear Search

Algorithm:



Step-1 : Read array A, Size N, and Key
Step2: Start from the first element of the array.
Step3: Compare $A[i]$ with Key.
Step4: If $A[i] == \text{Key}$, Print the Position
Step5: If $A[i] \neq \text{Key}$, move to the next element.
Step6: Repeat the comparison until the key is found.
the array ends.
Step7: If the key is not found, Print "Element not found"
Step8: End.

Program:

```
#include<iostream>
using namespace std;

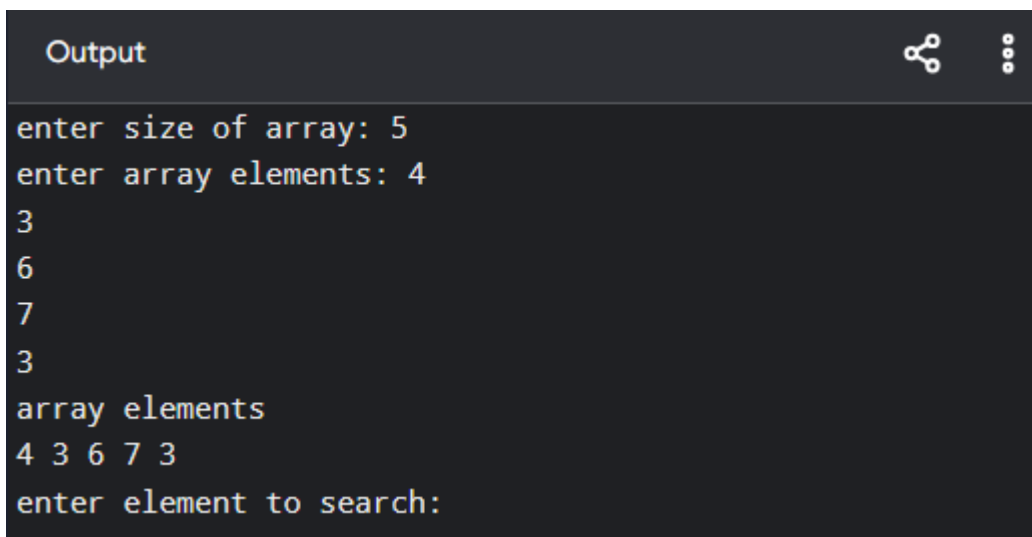
class linearsearch {
public:
    int arr[100], size, key;

    void arrayinput() {
        cout << "enter size of array: ";
        cin >> size;
        cout << "enter array elements: ";
        int i;
        for(i = 0; i < size; i++) {
            cin >> arr[i];
        }
    }

    void printarray() {
        int i;
        Name: Yaswanth Sai
        Enrollment Number:92400118993
```

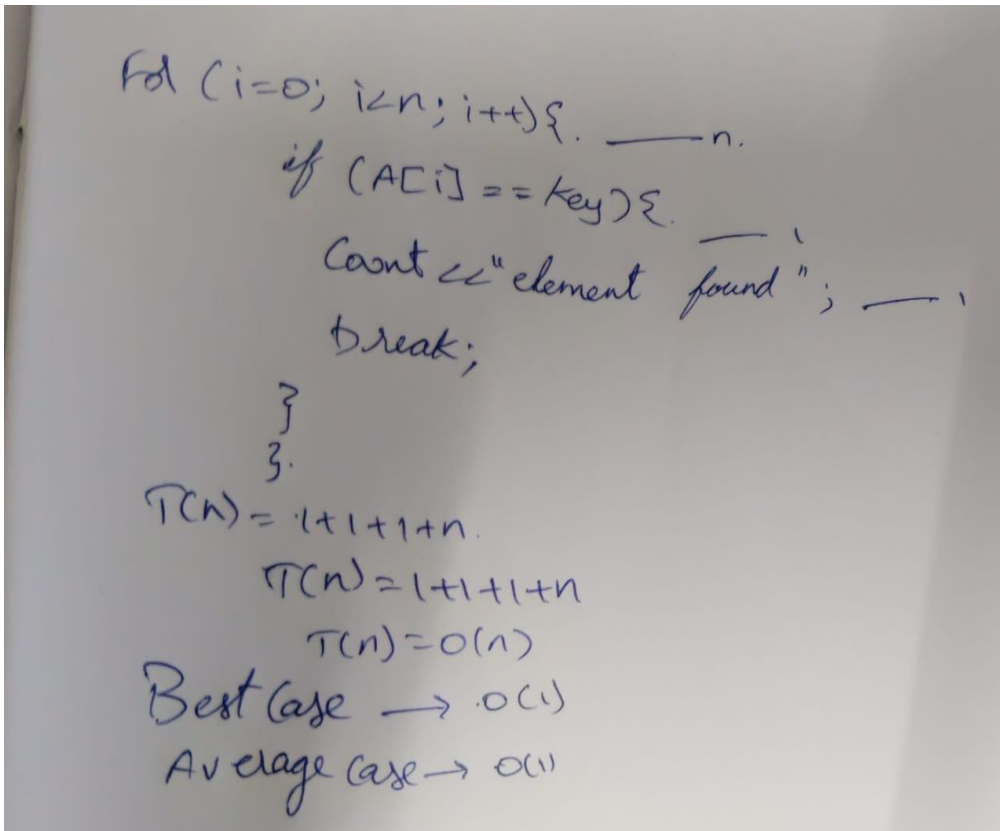
```
for(i = 0; i < size; i++) {  
    cout << arr[i] << " ";  
}  
  
cout << endl;  
}  
  
void searchlogic() {  
    int i;  
    for(i = 0; i < size; i++) {  
        if(arr[i] == key) {  
            cout << "element found at index " << i << endl;  
            break;  
        }  
    }  
    if(i == size) {  
        cout << "element not found" << endl;  
    }  
}  
};  
  
int main() {  
    linearsearch ls;  
    ls.arrayinput();  
    cout << "array elements" << endl;  
    ls.printarray();  
    cout << "enter element to search: ";  
    cin >> ls.key;  
    ls.searchlogic();  
    return 0;  
}
```

Output:



```
Output  
enter size of array: 5  
enter array elements: 4  
3  
6  
7  
3  
array elements  
4 3 6 7 3  
enter element to search:
```

Time Complexity:

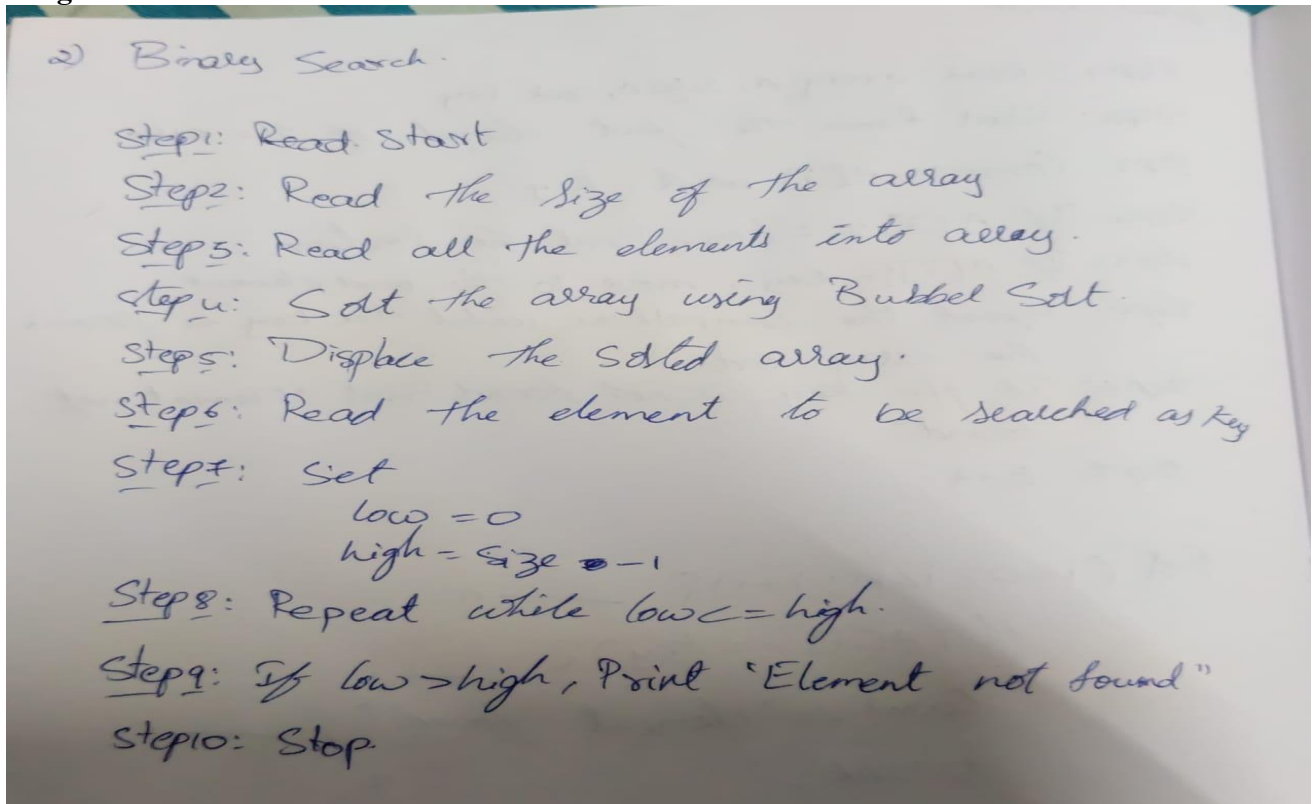


```
for (i=0; i<n; i++) {  
    if (A[i] == key) {  
        cout << "element found";  
        break;  
    }  
}
```

$T(n) = 1 + 1 + 1 + n$
 $T(n) = 1 + 1 + 1 + n$
 $T(n) = O(n)$
Best case $\rightarrow O(1)$
Average case $\rightarrow O(n)$

2) Binary Search:

Algorithm:



Program:

```
#include <iostream>
using namespace std;
class BinarySearch
{
public:
    int arr[100], n, key;
    void arrayInput()
    {
        cout << "Enter number of elements: ";
        cin >> n;
        cout << "Enter the sorted elements: ";
        for (int i = 0; i < n; i++)
        {
            cin >> arr[i];
        }
        cout << "Enter the element to search: ";
        cin >> key;
    }

    void printArray()
    {

```

```
for (int i = 0; i < n; i++)
{
    cout << arr[i] << " ";
}
cout << endl;
}

void search()
{
    int low = 0;
    int high = n - 1;

    while (low <= high)
    {
        int mid = (low + high) / 2;

        if (arr[mid] == key)
        {
            cout << "Element found at index " << mid << endl;
            return;
        }
        else if (key < arr[mid])
        {
            high = mid - 1;
        }
        else
        {
            low = mid + 1;
        }
    }

    cout << "Element not found." << endl;
}

};

int main()
{
    BinarySearch bs;
    bs.arrayInput();
    cout << "\nArray: ";
    bs.printArray();
    bs.search();
    return 0;
}
```



Output:

```
Output
Enter number of elements: 5
Enter the sorted elements: 3
4
7
6
2
Enter the element to search: 4

Array: 3 4 7 6 2
Element found at index 1
```

Time complexity:

Binary Search (A[], n, key) Σ — $T(n)$
low = 0; — (1)
high = n - 1; — (1)
while (low <= high) { — (1)
 mid = $\frac{low + high}{2}$; — $T(\frac{n}{2})$
 if (A[mid] == key) { — (1)
 return mid;
 }
 else if (key < A[mid]) { — (1)
 high = mid - 1;
 }
 else if (key > A[mid]) { — (1)
 low = mid + 1;
 }
}



$$\begin{aligned}
 &\{ \\
 &\{ \\
 &\text{return } -1 \\
 &\{ \\
 &T(n) = T(n/2) + 1 \\
 &\quad n=1 \\
 &T(n) = \begin{cases} 1 & n=1 \\ T(n/2) + 1 & n>1 \end{cases} \\
 &T(n) = T(n/2) + 1 \\
 &\quad n \rightarrow n/2 \\
 &T(n) = T(n/4) + 1 + 1 \\
 &\quad = T(n/4) + 2 \\
 &\quad n \rightarrow n/4 \\
 &T(n) = T(n/8) + 2 + 1 \\
 &\quad = T(n/8) + 3 \\
 &T(n) = T(n/2^k) + 3 \\
 &\quad = T\left(\frac{n}{2^k}\right) + k
 \end{aligned}$$

$$\begin{aligned}
 &n=1 \\
 &\frac{n}{2}=1 \\
 &n=2^k \\
 &\log n = \log 2^k \\
 &\log n = k \log 2 \\
 &k = \frac{\log n}{\log 2} \\
 &k = \log_2 n \\
 &T\left(\frac{n}{n}\right) \neq \log_2 n \\
 &T(1) \neq \log_2 n \\
 &\quad = 1 + \log_2 n \\
 &T(n) = O(\log_2 n) \rightarrow \text{in.} \\
 &\text{worst case} \\
 &\text{Best case} \rightarrow O(1) \\
 &\text{Average case} \rightarrow O(\log n)
 \end{aligned}$$

Conclusion:

The implementation of Linear Search and Binary Search demonstrates how different searching techniques work and how their performance varies. Linear Search is easy to implement and can be used on unsorted arrays, while Binary Search provides faster searching for sorted arrays. The time analysis shows that Binary Search is more efficient for large sorted datasets due to its logarithmic time complexity.